

Projet DevOps

Infrastructure automatisée — M2

Proxmox · Terraform · Ansible · Kubernetes

Jérémy FOURNIER — Kenza NAIT ATMANE

Master 2 Réseaux et Télécommunications | 2026

Proxmox

Terraform

Ansible

Kubernetes

Wiki.js

Sommaire

Plan de la présentation

01 Contexte & Objectifs DevOps

03 Infrastructure as Code — Terraform

05 Architecture Kubernetes — Cluster & Composants

07 Déploiement Applicatif — Wiki.js & Scaling

02 Hyperviseur Proxmox

04 Configuration & Sécurité — Ansible

06 Services Plateforme — MetalLB & Ingress

08 Workflow Complet & Conclusion

01

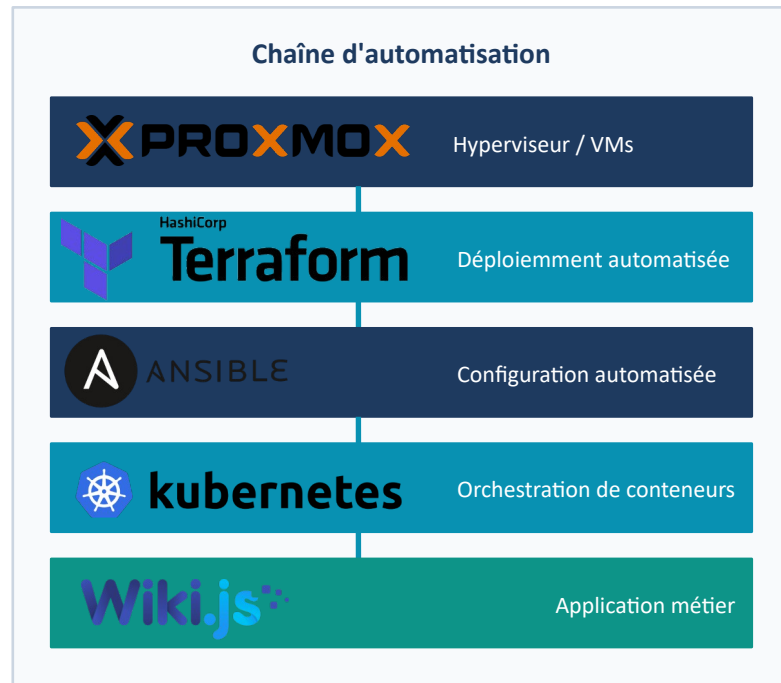
Contexte & Objectifs DevOps

Problématique & Objectifs du projet

Pourquoi cette approche DevOps ?

Les infrastructures modernes ne peuvent plus être gérées manuellement. Le projet vise à construire une **chaîne d'automatisation** complète et **reproductible**, de la création des VMs jusqu'à l'application en production.

- ▶ **Provisionnement automatique** des ressources système
- ▶ **Configuration homogène** et sécurisée des machines
- ▶ Déploiement applicatif **conteneurisé** et **orchestré**
- ▶ Montée en charge **dynamique** selon la demande
- ▶ Infrastructure **déclarative** — 100% reproductible



02

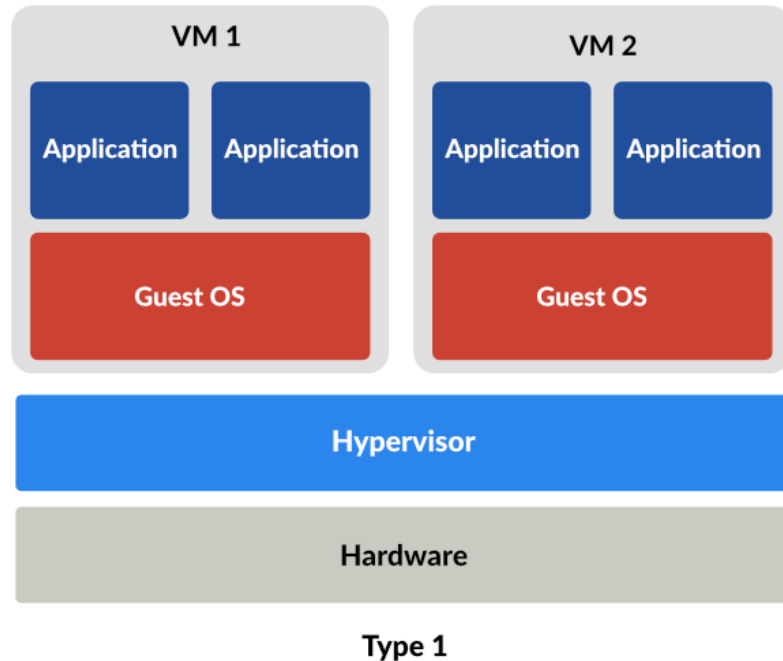
Hyperviseur Proxmox

Hyperviseur Proxmox VE

Type 1 bare-metal — socle de l'infrastructure

Approche retenue :

- ▶ Hyperviseur de Type 1 : s'exécute directement sur le matériel physique, sans OS intermédiaire
- ▶ Basé sur KVM (virtualisation matérielle) + LXC
- ▶ Interface web + API REST complète → intégration Terraform native
- ▶ Open source — alternative souveraine à VMware
- ▶ Gestion native : VMs, stockage, réseau, snapshots, Cloud-Init



Golden Image — Ubuntu 24.04 Cloud

Pourquoi une Cloud Image plutôt qu'une ISO classique ?

Approche retenue :

- ▶ Image Cloud officielle Ubuntu 24.04 LTS (.img) — ~350 Mo
- ▶ Cloud-Init intégré : IP, SSH, user injectés au boot par Terraform
- ▶ Socle systemd + glibc + cgroups v2 → compatibilité K8s native
- ▶ Ansible devient la seule "source de vérité" — pas de config dans l'image
- ▶ Lutte contre le Configuration Drift : tout est dans les playbooks

Pipeline de création du template :



Comparaison OS

OS	K8s	Cloud-Init
Alpine Linux	⚠ Faible	⚠ Partiel
Debian 12	✅ Excellent	✅ Natif
Ubuntu 24 ✓	✅ Standard	✅ Optimisé
Rocky Linux	✅ Très bon	✅ Natif

03

Infrastructure as Code — Terraform

Infrastructure as Code — Les 3 piliers

Terraform / OpenTofu : décrire l'infrastructure comme du code



Idempotence

- Exécuter le même script N fois → résultat toujours identique
- Si l'état cible est déjà atteint, aucune action n'est effectuée



Déclaratif

- On décrit CE QUE l'on veut, pas COMMENT le faire
- Le fichier HCL définit l'état final souhaité
- Terraform calcule lui-même les actions nécessaires pour y parvenir

```
# Déploiement de 4 VMs en boucle depuis le template Ubuntu
resource "proxmox_virtual_environment_vm" "vm" {
  count    = local.total_vms          # master + workers + db
  name     = "tf-k8s-node-${count.index}"
  node_name = var.proxmox_node

  # Clonage du template Golden Image
  clone {
    vm_id = var.template_vmid
    full  = true
  }

  # Ressources matérielles
  cpu    { cores = 2; type = "host" }
  memory { dedicated = 8192 }
  disk   { datastore_id = "local-lvm"; interface = "scsi0"; size = 30 }

  # Cloud-Init : injection réseau + utilisateur + clé SSH
  initialization {
    user_account {
      username = "ansible"
      keys     = [var.ssh_public_key]
    }
  }
  ip_config {
    ipv4 {
      address = "${var.vm_ip_base}${var.vm_ip_start + count.index}/24"
      gateway = var.vm_gateway
    }
  }
}
```

Terraform — Ce que l'on déploie

Provisionnement de 4 VMs Ubuntu via Proxmox API

Approche retenue :

- ▶ Provider BPG (bpg/proxmox) — API Proxmox complète
- ▶ Clonage du template Ubuntu 24.04 → 4 VMs
- ▶ Cloud-Init injecte : IP statique, user ansible, clé SSH
- ▶ Inventaire Ansible généré automatiquement (templatefile)
- ▶ terraform apply → 4 VMs prêtes en ~1 minute

Cycle de vie :



VMs provisionnées par Terraform

tf-k8s-master

192.168.122.150 — K8s Control Plane

tf-k8s-node-1

192.168.122.151 — K8s Worker Node

tf-k8s-node-2

192.168.122.152 — K8s Worker Node

tf-db-postgres

192.168.122.153 — Base PostgreSQL

04

Configuration & Sécurité — Ansible

Ansible — Automatisation de la configuration

Agentless · YAML · Idempotent

Approche retenue :

- ▶ Agentless : connexion SSH uniquement, rien à installer sur les VMs
- ▶ Inventaire généré par Terraform → cohérence totale
- ▶ Playbooks numérotés et ordonnés → dépendances explicites
- ▶ Chaque playbook correspond à une brique fonctionnelle précise
- ▶ Idempotent : safe à rejouer sans effet de bord

Terraform → provisionnement (VMs nues) Ansible → configuration (logiciel)

Les 8 playbooks

1 k8s-master

2 k8s-workers

3 db-postgres

4 metallb

5 ingress-nginx

6 metrics-server

7 wikijs-init

8 wikijs-scale

Hardening CIS Ubuntu 24.04

Sécurisation selon le référentiel CIS Benchmark Niveau 1

Approche retenue :

- ▶ CIS Benchmark : référentiel industriel de bonnes pratiques sécurité
- ▶ Désactivation des services inutiles (Bluetooth, Avahi...)
- ▶ Paramètres noyau sécurisés via sysctl
- ▶ SSH : root login désactivé, authentification par clé uniquement
- ▶ Audit des logs activé (auditd), permissions fichiers sensibles vérifiées
- ▶ Audit automatisé via GOSS — rapport par VM avec historique

Résultat : passage de 48,7 % → 88,0 % de conformité CIS sur toutes les VMs

Résultats Audit CIS (par VM)

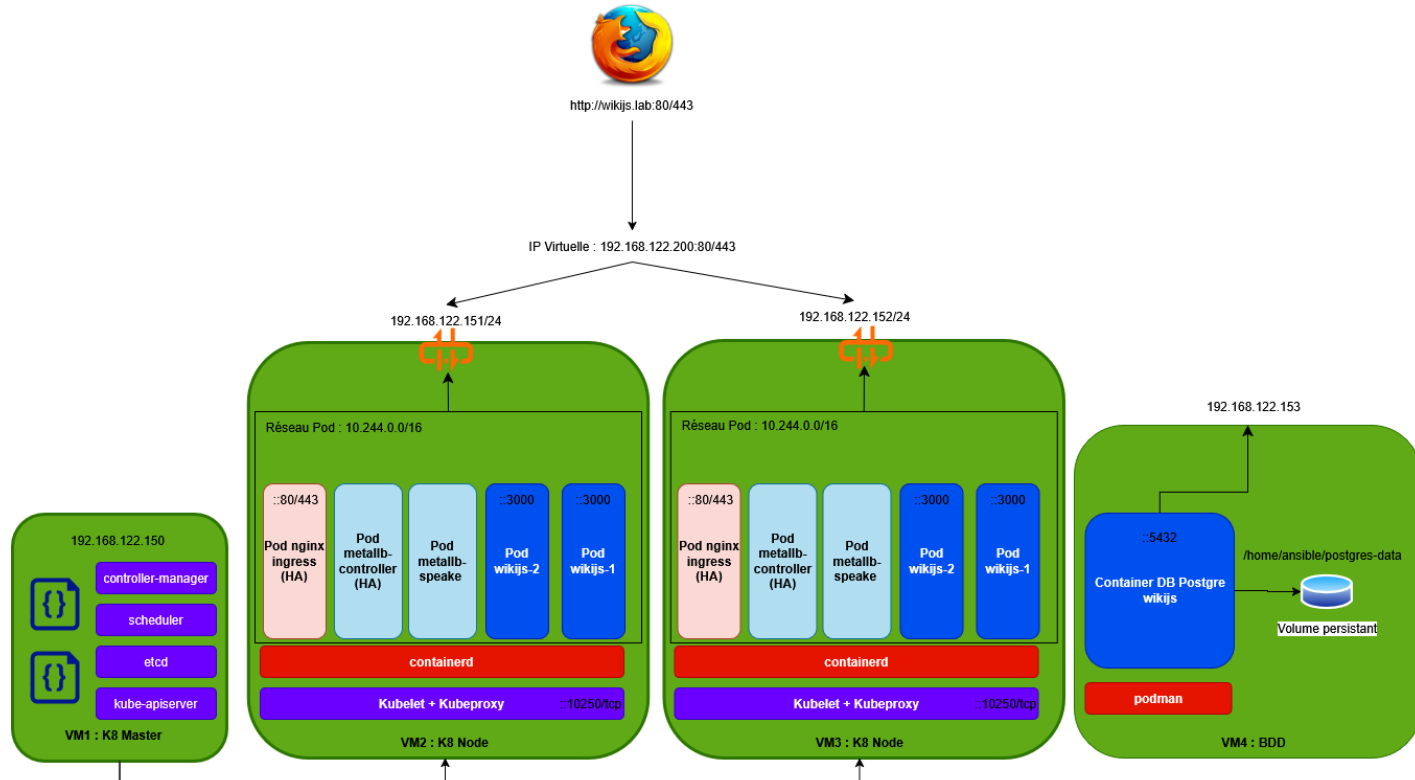
Critère	Avant	Après
Failed	338	74
Success	368	665
Skipped	50	17
Total	756	756
Conformité	48,7 %	88,0 %

05

Architecture Kubernetes — Cluster & Composants

Architecture du cluster Kubernetes

Control Plane vs Workers — qui fait quoi ?



Réseau des pods — Calico (CNI)

Container Network Interface : pourquoi et comment ?

Qu'est-ce que le CNI ?

- Les **pods de nœuds différents** ne peuvent pas communiquer
- Le CNI **attribue une IP unique à chaque pod** et configure le **routage inter-nœuds**
- Règle fondamentale : tout pod peut parler à tout pod, **sans NAT interne**



>



flannel

Configuration dans ce projet

- ▶ Réseau des pods : 10.244.0.0/16
- ▶ Réseau des VMs : 192.168.122.0/24
- ▶ Mode transport : VXLAN
- ▶ NAT activé : les pods sortent sur l'IP de leur VM hôte

Workflow de déploiement du cluster Kubernetes

Les 8 playbooks Ansible — séquence ordonnée

1

Init Master

kubeadm init, containerd, Calico CNI, kubeconfig

3

PostgreSQL

Podman rootless sur VM dédiée (hors cluster)

5

Ingress NGINX

Reverse proxy HTTP exposé via MetalLB (Kustomize)

7

Wiki.js Init

Helm install, 1 replica, assistant de configuration

2

Join Workers

kubeadm join, cluster distribué opérationnel

4

MetalLB

LoadBalancer on-premise, pool IP 192.168.122.200-220

6

Metrics Server

Collecte CPU/RAM → prérequis HPA + kubectl top

8

Wiki.js Scale

Helm upgrade, 2+ replicas, resources, HPA actif

06

Services Plateforme — MetalLB & Ingress NGINX

MetalLB — LoadBalancer on-premise

Résoudre le problème de l'IP externe

Le besoin :

- Le client doit accéder à l'application via **une URL fixe** (wikijs.lab)
- Cette URL doit pointer vers une **IP stable**, indépendante des workers
- MetalLB crée cette **IP virtuelle** — équivalent réseau de VRRP/HSRP
- Un nœud worker est élu "**speaker**" : il répond aux **requêtes ARP** pour cette IP
- Si ce worker tombe, un autre **prend le relais automatiquement**

Configuration dans ce projet

- ▶ IP Virtuelle attribuée : 192.168.122.200
- ▶ Pool disponible : 192.168.122.200-220
- ▶ Mode : L2 / ARP
- ▶ Relié à : Service Ingress NGINX Controller

Ingress NGINX — Routage HTTP centralisé

Une seule IP, plusieurs applications — routage par nom de domaine

Approche retenue :

- ▶ Un objet Ingress = une règle déclarative (host → service)
- ▶ Le contrôleur NGINX lit les Ingress et configure le reverse proxy dynamiquement
- ▶ Premier composant de la chaîne à comprendre le protocole HTTP (L7)
- ▶ Déployé via Kustomize : 2 patches sur le manifeste upstream officiel
- ▶ 2 réplicas → haute disponibilité (un par worker)

Objet Ingress pour Wiki.js :

```
host: wikijs.lab → service: wikijs → port: 80
```

Chaîne de routage

MetalLB 192.168.122.200
[L2/L3]

Service LB ingress-nginx-controller
[L4 TCP]

NGINX Pod Routage HTTP Host:
[L7 HTTP]

Service K8s wikijs (ClusterIP)
[L4]

Pod Wiki.js Réponse HTTP
[App]

07

Déploiement Applicatif — Wiki.js & Scaling

Helm — Gestionnaire de paquets Kubernetes

L'équivalent d'apt/yum pour Kubernetes

Helm package un ensemble de manifestes Kubernetes en un Chart réutilisable, avec un fichier de configuration centralisé (values.yaml) et une gestion complète du cycle de vie.

Centralisation

Tous les paramètres dans un seul values.yaml

Standardisation

Charts officiels maintenus par la communauté

Versioning

Chaque déploiement est versionné → rollback facile

Templating

Variables dynamiques dans les manifestes YAML

Commandes Helm utilisées

```
$ helm repo add requarks  
https://charts.js.wiki  
  
$ helm install wikijs requarks/wiki  
-f values-wikijs-init.yaml  
-n wikijs  
  
$ helm upgrade wikijs requarks/wiki  
-f values-wikijs-scale.yaml  
-n wikijs  
  
$ helm list -n wikijs  
$ helm history wikijs -n wikijs
```

Wiki.js — Déploiement en deux phases

Contrainte d'initialisation unique — pattern industriel

Wiki.js écrit sa configuration initiale en base de données lors du premier démarrage. Si plusieurs instances démarrent simultanément, des conflits d'état apparaissent. La solution : séparer init et scale.

Phase 1 — Initialisation (playbook 7)

- ▶ helm install — 1 seul replica
- ▶ Connexion PostgreSQL configurée (IP externe)
- ▶ Ingress exposé : `http://wikijs.lab`
- ▶ Setup wizard en navigateur (admin + URL)

Phase 2 — Mise à l'échelle (playbook 8)

- ▶ helm upgrade — 2 réplicas + ressources définies
- ▶ requests: 200m CPU / 256Mi RAM
- ▶ limits: 500m CPU / 768Mi RAM
- ▶ HPA activé : scale automatique selon CPU

HPA : `minReplicas=2 · maxReplicas=5 · targetCPUUtilizationPercentage=70%` — API autoscaling/v2

08

Workflow Complet & Conclusion

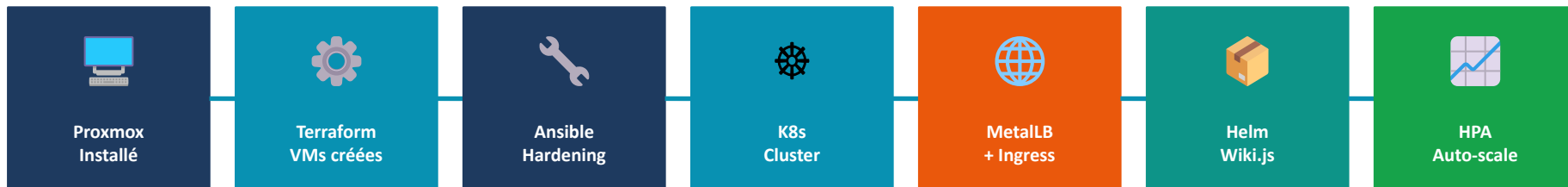
Flux de requête — Du navigateur au pod Wiki.js

Chaque composant agit dans son périmètre sans connaissance globale

1	Navigateur http://wikijs.lab	Résolution DNS /etc/hosts → 192.168.122.200	—
2	MetalLB 192.168.122.200	IP virtuelle annoncée en ARP. Niveau L2/L3 uniquement. Ne connaît pas HTTP.	L3
3	Service LoadBalancer ingress-nginx-controller	Répartit le trafic TCP vers les pods NGINX (2 réplicas). Niveau L4.	L4
4	NGINX Ingress Host: wikijs.lab	1er composant HTTP. Lit la règle Ingress → redirige vers Service wikijs.	L7
5	Service ClusterIP wikijs	Adresse interne stable. Équilibrage vers les pods Wiki.js actifs via iptables.	L4
6	Pod Wiki.js Conteneur applicatif	Traite la requête HTTP et retourne la réponse au client.	App

Workflow complet de déploiement

De zéro à l'application en production — pipeline entièrement automatisé



Composant	État	Détail
Cluster K8s	✓ Running	1 master + 2 workers — kubeadm
PostgreSQL	✓ Running	Podman rootless, VM dédiée .153
MetalLB + Ingress	✓ Running	IP 192.168.122.200, 2 réplcas NGINX
Wiki.js	✓ Running	2 pods min, HPA actif (max 5)
Hardening CIS	✓ 88%	Avant : 48,7% — Après : 88,0%

Bilan & Perspectives

- 1 Infrastructure 100% automatisée — de la VM au pod en production sans intervention manuelle
- 2 Séparation claire des responsabilités entre chaque outil de la chaîne
- 3 Sécurité intégrée dès la conception — Hardening CIS 88% + containerd rootless
- 4 Scalabilité native — HPA ajuste automatiquement les ressources selon la charge CPU
- 5 Reproductibilité totale — tout est versionné et rejouable depuis le code

Perspectives : CI/CD pipeline (GitHub Actions → cluster), cert-manager TLS, Prometheus + Grafana, etcd multi-nœuds HA



Merci de votre attention

Des questions ?

Jérémy FOURNIER · Kenza NAIT ATMANE | M2 — 2026